

ABTalksOnAI

Global Coding Challenge (Season-2)

Day-07 & Day-08

Weekly Review – NLP Workflow

1. Imported Required Libraries

```
import re
import numpy as np
import nltk

from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer

from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics.pairwise import cosine_similarity

import matplotlib.pyplot as plt

nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
```

2. Sample Dataset (News / Reviews)

```
dataset = [
    "Artificial intelligence is transforming the world.",
    "Machine learning and AI are widely used in modern technology.",
    "Natural language processing enables computers to understand human language.",
    "AI applications include healthcare, finance, and robotics."
]
```

3. Text Cleaning Function

```
def clean_text(text):

    text = text.lower()
    text = re.sub(r'^a-z\s', '', text)

    return text
```

4. Tokenization

```
def tokenize_text(text):  
  
    tokens = word_tokenize(text)  
  
    return tokens
```

5. Stopword Removal

```
stop_words = set(stopwords.words('english'))  
  
def remove_stopwords(tokens):  
  
    filtered_words = [word for word in tokens if word not in stop_words]  
  
    return filtered_words
```

6. Stemming

Using Porter Stemmer.

```
stemmer = PorterStemmer()  
  
def apply_stemming(tokens):  
  
    stemmed_words = [stemmer.stem(word) for word in tokens]  
    return stemmed_words
```

7. Lemmatization

```
lemmatizer = WordNetLemmatizer()  
  
def apply_lemmatization(tokens):  
  
    lemmatized_words = [lemmatizer.lemmatize(word) for word in tokens]  
  
    return lemmatized_words
```

8. Full NLP Preprocessing Pipeline

This pipeline combines all preprocessing steps.

```
def preprocess_pipeline(text):  
    cleaned = clean_text(text)  
    tokens = tokenize_text(cleaned)  
    filtered = remove_stopwords(tokens)  
    stemmed = apply_stemming(filtered)  
    lemmatized = apply_lemmatization(stemmed)  
    return lemmatized
```

9. Applied Pipeline to Dataset

```
processed_dataset = []  
  
for text in dataset:  
    processed_dataset.append(preprocess_pipeline(text))  
  
print(processed_dataset)
```

10. Word Frequency Distribution

Using NumPy for counting.

```
all_words = []  
  
for sentence in processed_dataset:  
    all_words.extend(sentence)  
  
unique_words, counts = np.unique(all_words, return_counts=True)  
  
word_freq = dict(zip(unique_words, counts))
```

```
print(word_freq)
```

11. Top 20 Most Frequent Words

```
sorted_words = sorted(word_freq.items(), key=lambda x: x[1], reverse=True)

top_words = sorted_words[:20]

words = [w[0] for w in top_words]
counts = [w[1] for w in top_words]

plt.bar(words, counts)
plt.xticks(rotation=45)
plt.title("Top Frequent Words")
plt.show()
```

12. Text Similarity Example

Using cosine similarity.

```
doc1 = dataset[0]
doc2 = dataset[1]

vectorizer = CountVectorizer()
vectors = vectorizer.fit_transform([doc1, doc2])
similarity = cosine_similarity(vectors)[0][1]

print("Cosine Similarity:", similarity)
```

13. Observations from Weekly Review

1. Importance of Text Cleaning

Removing punctuation, numbers, and irrelevant tokens significantly improves data quality.

2. Tokenization as the First NLP Step

Tokenization converts raw text into manageable word units for further processing.

3. Stopword Removal Reduces Noise

Common words like "the", "is", and "and" appear frequently but contribute little meaning.

4. Stemming vs Lemmatization

Stemming reduces words aggressively while lemmatization produces more meaningful base words.

5. Preprocessing Pipelines Improve Workflow

Creating a reusable pipeline function makes preprocessing consistent and scalable.

6. Word Frequency Helps Identify Key Terms

Frequency distributions reveal important words in datasets and are useful for text analytics.

Final Review Conclusion

This weekly review reinforced the complete NLP preprocessing workflow including text cleaning, tokenization, normalization, frequency analysis, and similarity measurement. Structuring preprocessing steps into reusable functions improves readability and ensures efficient NLP pipeline design.